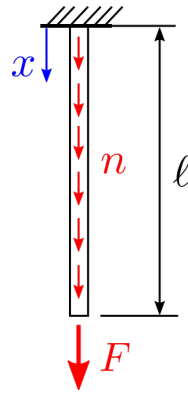


Notebook_StabFEM

May 16, 2025

1 Der Dehnstab

In diesem Notebook wird die FEM-Implementierung für den Dehnstab erläutert und erprobt.



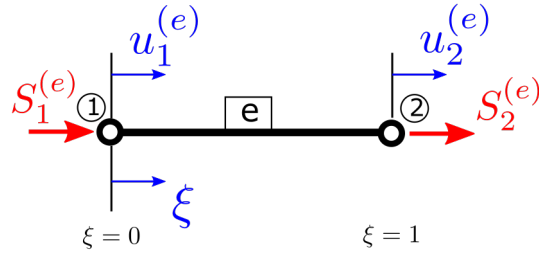
Die Differentialgleichung des Dehnstabes lautet:

$$EAu'' = -n(x)A ,$$

Für diese Differentialgleichung haben wir in der Vorlesung die schwache Form hergeleitet:

$$\int_0^\ell \delta \epsilon \cdot E \epsilon A \, dx - \int_0^\ell \delta u \cdot n A \, dx - \delta u(\ell) \cdot F = 0$$

Ein Finites Element besteht in dem vorliegenden Beispiel aus 2 Knoten. Sowohl die Knoten, als auch die Elemente werden im Allgemeinen nummeriert, damit man sie eindeutig ansprechen kann. Im Bild sind die Elementnummern in den rechteckigen Kästen neben dem Element eingetragen. Die Knotennummern sind in den Kreisen neben den Knoten dargestellt. Die Knoten sind die Träger der primären Feldvariablen (hier: Verschiebung u) und Ziel der Finiten Elemente Berechnung ist die Bestimmung der primären Variablen, auch Freiheitsgrade (English: Degree of freedom **Dof**) an den Knoten.



Auf diesen Elementen werden dann einfache Ansatzfunktionen verwendet, welche eine lineare Approximation des Verschiebungsfeldes darstellen:

$$N_1(\xi) = (1 - \xi) \quad N_2(\xi) = \xi \quad (1)$$

$$u_h(\xi) = N_1(\xi)\hat{u}_1^{(e)} + N_2(\xi)\hat{u}_2^{(e)} \quad (2)$$

$$u_h(\xi) = \sum_{I=1}^2 N_I(\xi)\hat{u}_I^{(e)} \quad (3)$$

```
[1]: import numpy as np
import plotly.graph_objects as go
```

```
# Define the x values
x = np.linspace(0, 1, 100)

N1 = 1-x
N2 = x
```

```
[2]: # Create the figure
import matplotlib.pyplot as plt

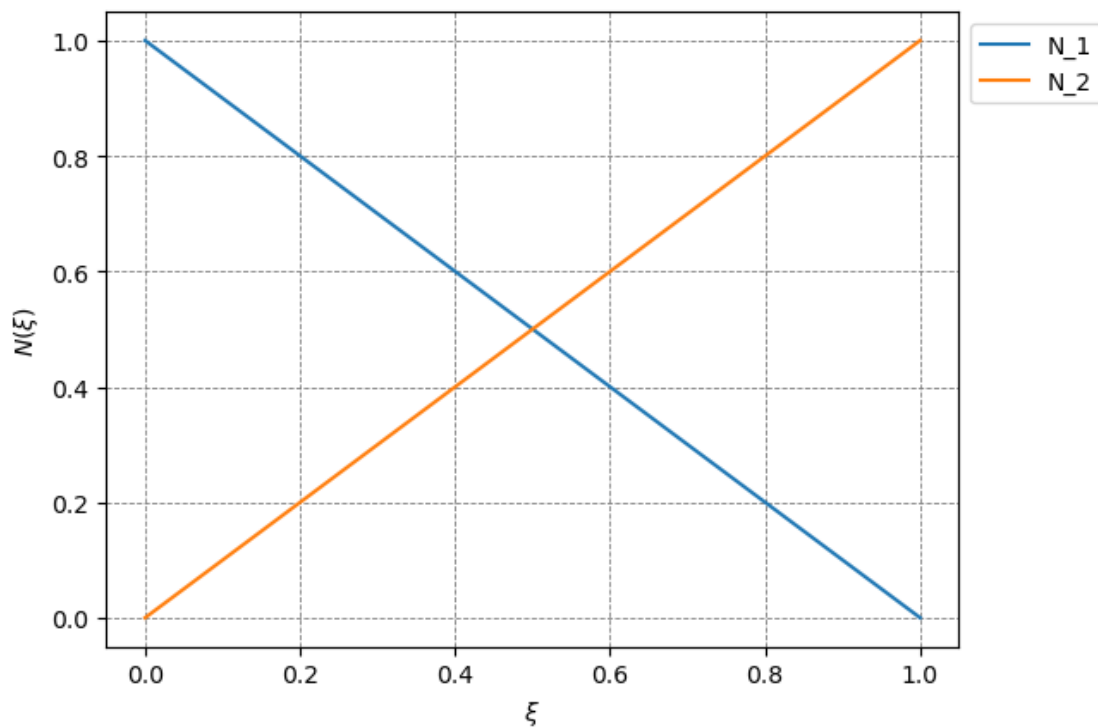
# Create the figure and axis
fig, ax = plt.subplots()

# Add traces for each shape function
ax.plot(x, N1, label='N_1')
ax.plot(x, N2, label='N_2')

# Update layout
ax.set_xlabel(r"$\xi$")
ax.set_ylabel(r"$N(\xi)$")
ax.legend(loc='upper left', bbox_to_anchor=(1, 1))
ax.grid(True, linestyle='--', color='grey', linewidth=0.6)

# Set font properties
plt.rc('font', family='serif', size=15)
```

```
# Show the plot
plt.show()
```



1.1 Die Steifigkeitsmatmatrix des Dehnstabes

Im nachfolgenden Python-Code wird die Steifigkeitsmatrix des Dehnstabes berechnet:

```
[3]: import sympy as sp
xi,ell,E,A = sp.symbols('xi, ell,E,A')

# Formfunktionen
N1 = 1-xi
N2 = xi
N = sp.Matrix([N1,N2])

# Ableitungen der Formfunktionen
dNdx = sp.diff(N,xi)

# Steifigkeitsmatrix
Kmat = sp.integrate(dNdx*dNdx.T,(xi,0,1))*E*A/ell
Kmat
```

```
[3]: 
$$\begin{bmatrix} \frac{AE}{\ell} & -\frac{AE}{\ell} \\ -\frac{AE}{\ell} & \frac{AE}{\ell} \end{bmatrix}$$

```

2 Analytische Lösung der DGL des Dehnstabes

Im folgenden wird die analytische Lösung der Differentialgleichung des Dehnstabes berechnet. Dafür wird die Differentialgleichung des Dehnstabes zweimal integriert. Die Integrationskonstanten werden durch die Randbedingungen bestimmt.

```
[4]: E,A,x,n,F = sp.symbols('E,A,x,n,F')
u = sp.Function('u')
solution = sp.dsolve(
    E*A*u(x).diff(x,2)+n*A,u(x) # DGL
    ,ics={ # Randbedingungen
        u(0):0, # Verschiebungsrandbedingung
        u(x).diff(x).subs({'x':ell}):F/E/A # Kraftrandbedingung
    }
)
ufun=solution.rhs
ufun
```

$$[4]: -\frac{nx^2}{2E} + \frac{x(Aln + F)}{AE}$$

Die Schnittgröße $N(x)$ erhält man dann aus:

$$\sigma = E\epsilon = E \frac{\partial u}{\partial x}$$

$$N(x) = \sigma A = EA \frac{\partial u}{\partial x}$$

```
[5]: Nfun = E*A*solution.rhs.diff(x)
Nfun
```

$$[5]: AE \left(-\frac{nx}{E} + \frac{Aln + F}{AE} \right)$$

3 FEM Implementierung des Dehnstabs

Das folgende ist nur ein Ausdruck der FEM Implementierung aus dem Python Modul `StabFEM.py` und dient nur der Veranschaulichung:

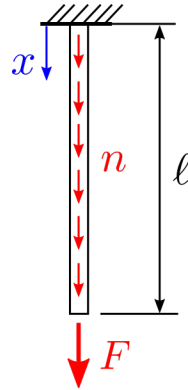
```
[6]: # from IPython.display import display, Markdown

# # Open the Python file and read its content
# with open('StabFEM.py', 'r') as file:
#     content = file.read()

# # Display the content as Markdown
# display(Markdown(f'``python\n{content}\n``'))
```

4 Beispiele

4.1 Beispiel 1:



Gegeben:

- $E = 210 \text{ GPa}$
- $A = 33.4 \text{ cm}^2$ (I-Profil - 200)
- $l = 4 \text{ m}$
- $F = 5 \text{ kN}$
- $n = 0.1 \text{ kN/m}$

Gesucht:

- $u(x=4\text{m})$
- $N(x=0\text{m})$
- Spannung an der Stelle $x=0\text{m}$

```
[7]: from StabFEM import StabFEM
import numpy as np
import matplotlib.pyplot as plt

#####
# SetUp des Problems
#####

bsp1 = StabFEM(numnp=2,numel=1)
X = np.array([
    [0,0],
    [4000.0,0]
])
elements = np.array([
    [0,1]
])
areas = [33.4*10**2]
youngsM = [210*10**3]
loads = [0.1]
```

```

# Setzen der Knotenkoordinaten, der Elementverbindungen und der Elementdaten
bsp1.setNodalCoordinates(X)
bsp1.setElementConnectivity(elements)
bsp1.setElementData(areas,youngsM,loads)

# Setzen der Randbedingungen
bsp1.setDirichletBoundaryCondition([0,0,1],[0,1,1],[0,0,0]) # Knoten, Richtung, Wert
bsp1.setExternalForces([1],[0],[5000]) # Knoten, Richtung, Wert

# Kontrolle der Eingaben
print(f"Knotenkoordinaten:\n{bsp1.coords}")
print(f"Elementverbindungen:\n{bsp1.elements}")
print(f"Elementdaten:\n{bsp1.eData}")

```

Knotenkoordinaten:

```
[[ 0.  0.]
 [4000. 0.]]
```

Elementverbindungen:

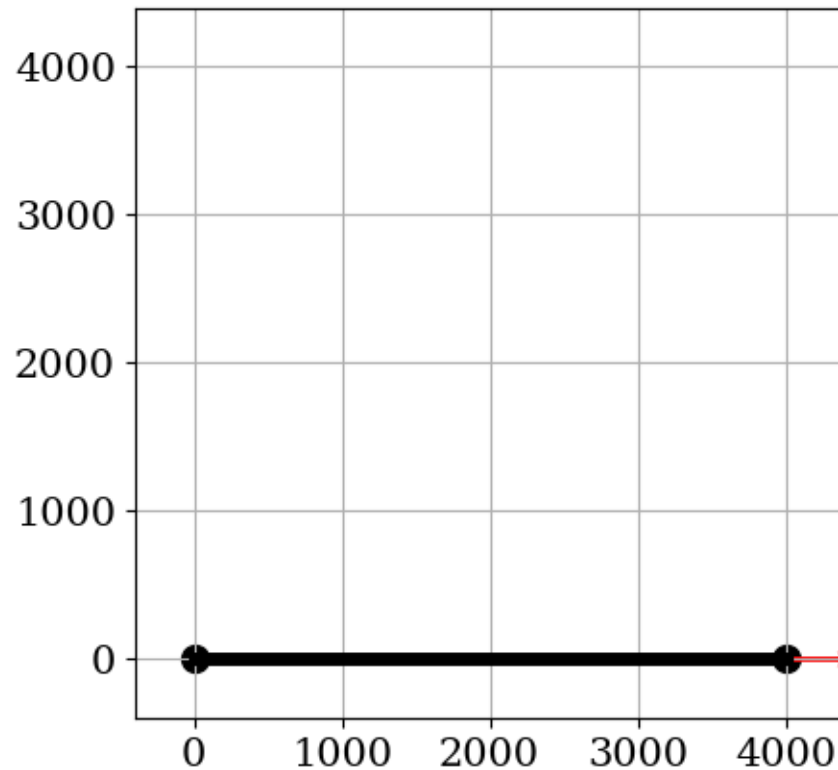
```
[[0 1]]
```

Elementdaten:

```
[{'area': 3340.0, 'youngsmodulus': 210000, 'lineLoad': 0.1}]
```

```
[8]: bsp1.plotMesh()
```

```
[8]: (<Figure size 640x480 with 1 Axes>, <Axes: >)
```



```
[9]: #####
# Lösen des Problems
#####

# Schleife über alle Elemente, bilden der Elementmatrizen und -vektoren und
# ↳assemblieren des globalen Systems
bsp1.assembleGlobalMatrix()
bsp1.assembleRightHandSide()
# Lösen des Gleichungssystems
bsp1.solveSystem()
display(r"$K_{ges}=$", (bsp1.Kges))
display("Displacement:", bsp1.dof)
display("Force:", bsp1.Fges)

'$K_{ges}=$'
array([[ 175350.,      0., -175350.,      0.],
       [      0.,      0.,      0.,      0.],
       [-175350.,      0.,  175350.,      0.],
       [      0.,      0.,      0.,      0.]])

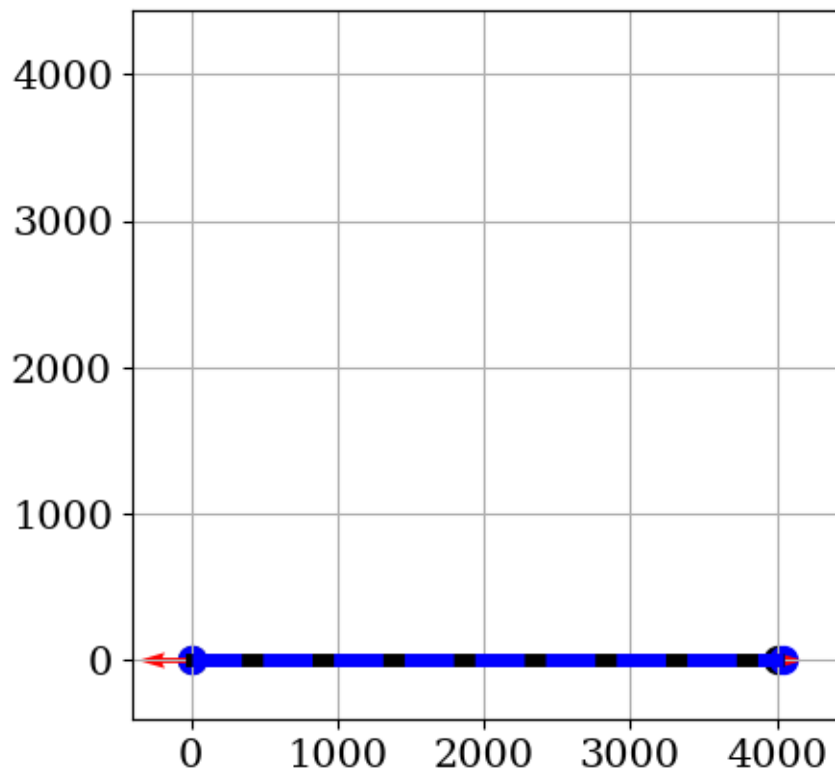
'Displacement:'
array([[0.,      0.],
```

```
[3.83803821, 0.      ]])
'Force:'
array([[ -1341000.,      0.],
       [  673000.,      0.]])
```

4.1.1 Darstellung der deformierten Struktur

```
[10]: bsp1.plotMesh(deformed = True, scale = 10)
```

```
[10]: (<Figure size 640x480 with 1 Axes>, <Axes: >)
```



4.1.2 Präsentation der Ergebnisse und Vergleich mit analytischer Lösung

Verschiebung am Ende des Balkens $u(x = \ell)$

```
[11]: print(f"Verschiebung am Ende des Balkens: {bsp1.getDisplacement(4000)} mm")
```

```
Verschiebung am Ende des Balkens: [3.83803821 0.      ] mm
```

Schnittkraft an der Einpannung $N(x = 0)$ Dies entspricht der Reaktionskraft:

```
[12]: print(f"Schnittkraft an der Einpannung: {-bsp1.Fges[0,0]/1000} kN")
```

Schnittkraft an der Einpannung: 1341.0 kN

4.1.3 Vergleich mit analytischer Lösung

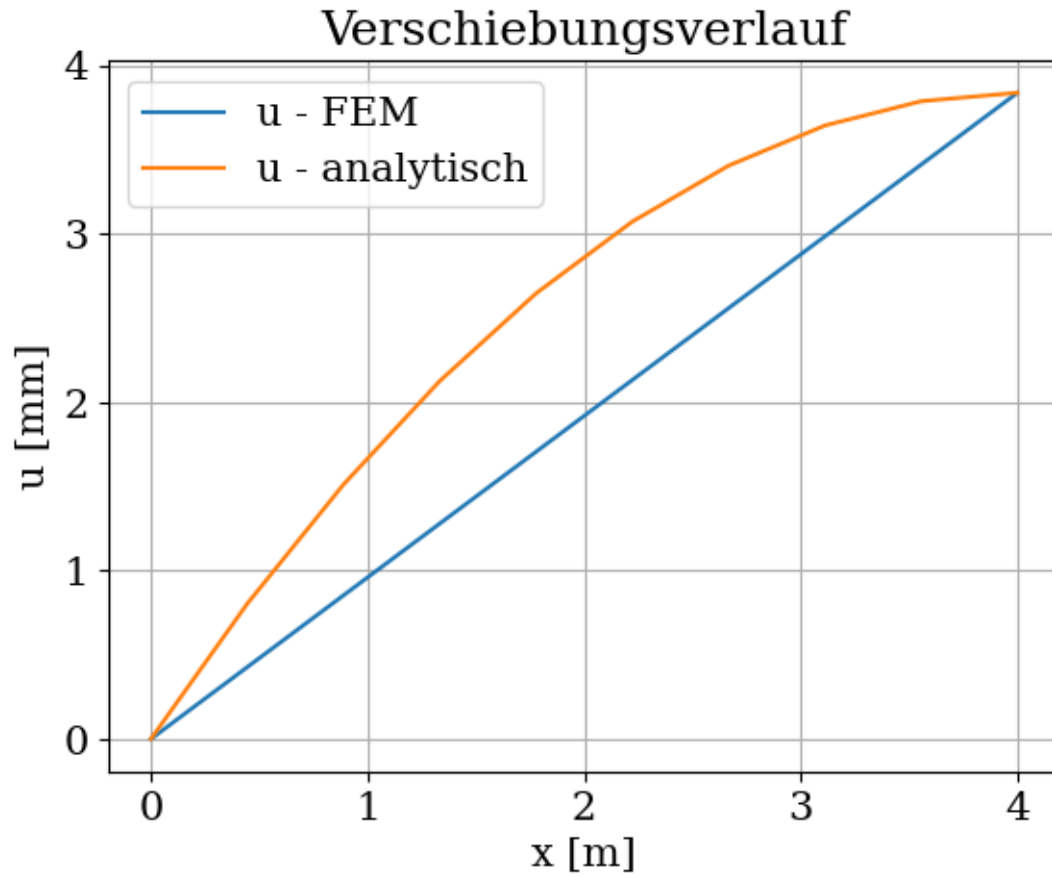
Verschiebungsverlauf $u(x)$

```
[13]: XN,u =bsp1.computeDisplacement()
fig,ax = plt.subplots(1,1)

ax.plot(XN/1000,u,label="u - FEM")
ax.set_title("Verschiebungsverlauf")
ax.set_xlabel("x [m]")
ax.set_ylabel("u [mm]")
ax.grid(True)

### Analytische Lösung
subs = {
    "E":bsp1.eData[0]["youngsmodulus"],
    "A":bsp1.eData[0]["area"],
    "n":bsp1.eData[0]["lineLoad"],
    "ell":4000,
    "F":5000
}
analytic_u = sp.lambdify(x,ufun.subs(subs),"numpy")
ax.plot(XN/1000,analytic_u(XN),label="u - analytisch")
ax.legend()
```

```
[13]: <matplotlib.legend.Legend at 0x7f4110b9c7c0>
```



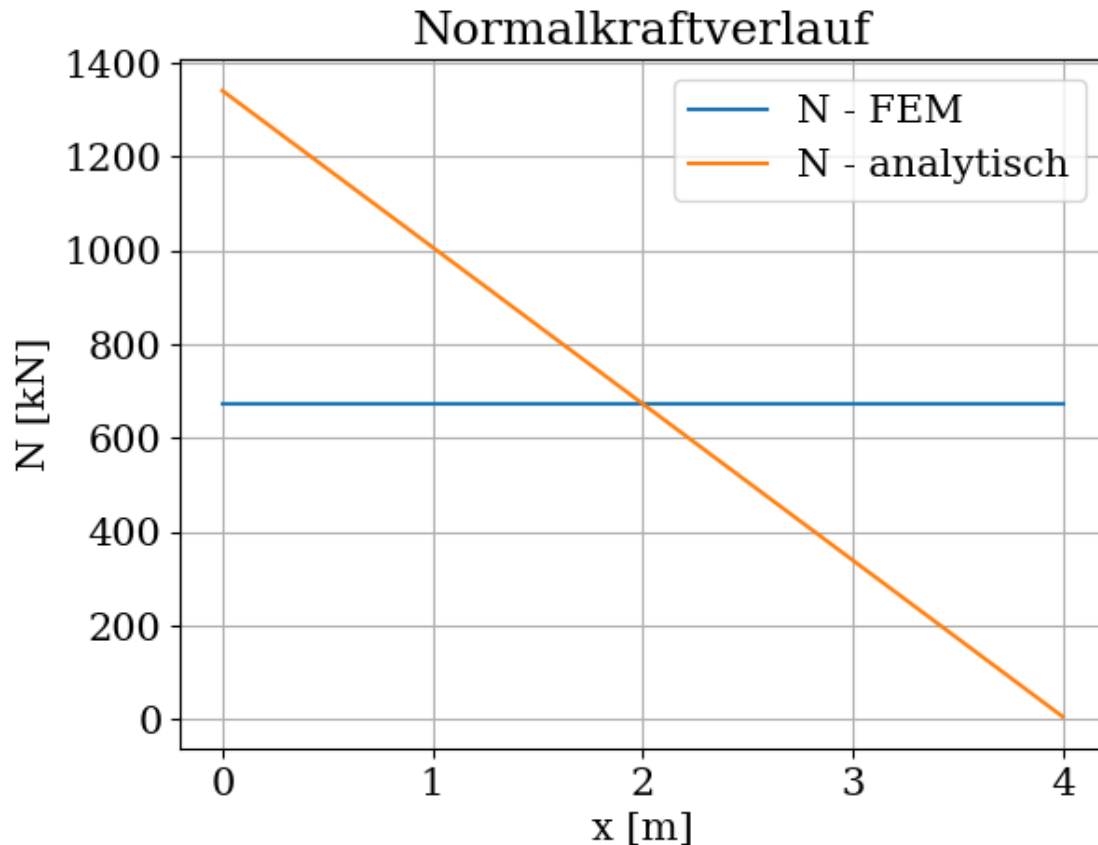
Normalkraftverlauf $N(x)$

```
[14]: XN,N =bsp1.computeNormalkraft()
fig,ax = plt.subplots(1,1)

ax.plot(XN/1000,N/1000,label="N - FEM")
ax.set_title("Normalkraftverlauf")
ax.set_xlabel("x [m]")
ax.set_ylabel("N [kN]")
ax.grid(True)

### Analytische Lösung
analytic_N = sp.lambdify(x,Nfun.subs(subs),"numpy")
ax.plot(XN/1000,analytic_N(XN)/1000,label="N - analytisch")
ax.legend()
```

[14]: <matplotlib.legend.Legend at 0x7f4110567f70>



```
[15]: errorN = (N[0]-analytic_N(0))/(analytic_N(0))
print(f"Fehler bei x=0: {errorN*100:.2f}%")
```

Fehler bei x=0: -49.81%

Spannungsverlauf $\sigma(x)$

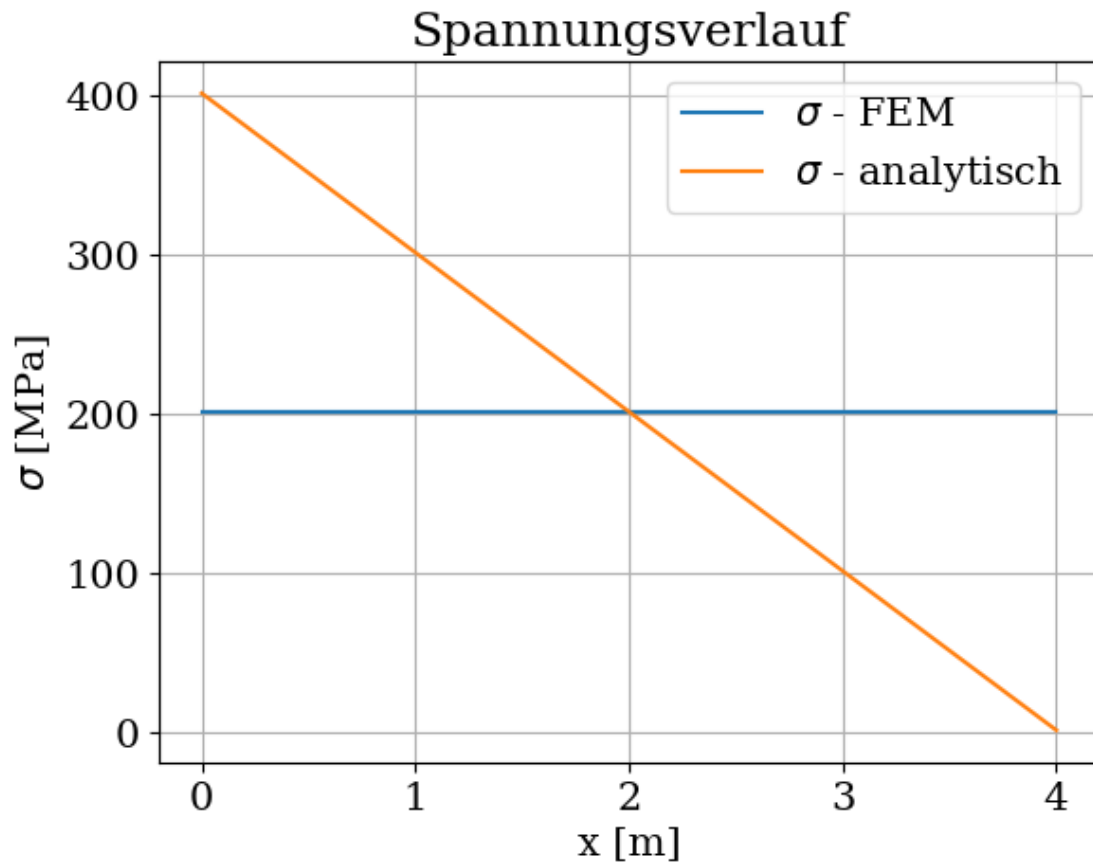
```
[16]: fig,ax = plt.subplots(1,1)

area = bsp1.eData[0]["area"]

ax.plot(XN/1000,N/area,label="$\\sigma$ - FEM")
ax.set_title("Spannungsverlauf")
ax.set_xlabel("x [m]")
ax.set_ylabel("$\\sigma$ [MPa]")
ax.grid(True)

### Analytische Lösung
analytic_N = sp.lambdify(x,Nfun.subs(subs),"numpy")
ax.plot(XN/1000,analytic_N(XN)/area,label="$\\sigma$ - analytisch")
ax.legend()
```

[16]: <matplotlib.legend.Legend at 0x7f4110593940>



4.1.4 Aufgabe:

Versuchen Sie das Problem mit 2 und mit 3 Elementen zu lösen. Ändern Sie hierfür den obigen Quellcode ab.

4.2 Beispiel 2 - Fehleranalyse

- Wieviele Elemente benötigt man, damit der Fehler der Normalkraft kleiner als 5 % ist?
- Tragen Sie den Fehler in Abhängigkeit von der Anzahl der Elemente in einem Diagramm auf
- Welcher Zusammenhang besteht zwischen dem Fehler und der relativen Elementgröße (Elementlänge) $\frac{\ell_e}{\ell}$?

```
[17]: def setUpProblem(numnp):  
    numel = numnp - 1  
    ell = 4000  
  
    Lastfall = StabFEM(numnp=numnp, numel=numel)
```

```

X1 = np.array([0.0,0.0])      #mm
X2 = np.array([ell,0.0])    #mm

# Generate interpolation points
t = np.linspace(0, 1, numnp)
X = X1 + t[:, np.newaxis] * (X2 - X1)
# print(f"Coordinates of the nodes: \n{X}")

Lastfall.setNodalCoordinates(X)

# Generate Elements
IX = np.column_stack((np.arange(0, numnp-1), np.arange(1, numnp)))
# print(f"Connectivity table: \n{IX}")

Lastfall.setElementConnectivity(IX)

# Material properties and cross section
Emod = 210000 # MPa
area = 33.4*10**2 # mm2
load = 0.1
Alist = [area for i in range(numel)]
Elist = [Emod for i in range(numel)]
loads = [load for i in range(numel)]

Lastfall.setElementData(Alist, Elist, loads)

# Dirichlet Randbedingungen (Verschiebung)
dnodes_y = [i for i in range(numnp)]
ddir_y = [1 for i in range(numnp)]
dval_y = [0.0 for i in range(numnp)]
dnodes = [0] + dnodes_y
ddir = [0] + ddir_y
dval = [0.0] + dval_y

Lastfall.setDirichletBoundaryCondition(dnodes,ddir,dval)

# Neumann Randbedingungen (Kraft)

Lastfall.setExternalForces([numnp-1],[0],[5000])

Lastfall.assembleGlobalMatrix()
Lastfall.assembleRightHandSide()
Lastfall.solveSystem()

```

```

    XN,N = Lastfall.computeNormalkraft()

    return N,XN,Lastfall

```

```

[18]: npoints = np.linspace(2,202,51)

NFEM = np.zeros((51,))
ErrorN= np.zeros((51,))
LClist = []
XNlist = []
NNlist = []

for i,p in enumerate(npoints):

    NN,XN,LC=setUpProblem(int(p))
    LClist.append(LC)
    XNlist.append(XN)
    NNlist.append(NN)
    NFEM[i]=NN[0]
    ErrorN[i] = np.abs((NN[0]-analytic_N(0))/(analytic_N(0)))*100

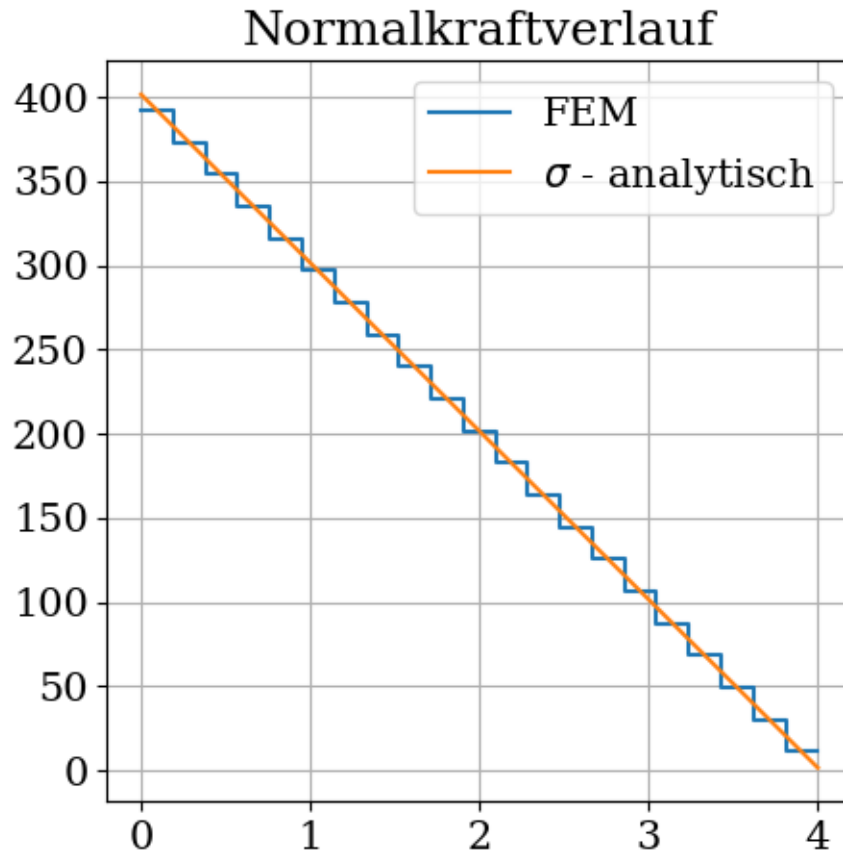
```

```

[19]: number = 5

fig, ax = plt.subplots(1,1,figsize=(5,5))
ax.plot(XNlist[number]/1000,NNlist[number]/area,label="FEM")
ax.set_title("Normalkraftverlauf")
ax.plot(XN/1000,analytic_N(XN)/area,label="$\\sigma$ - analytisch")
ax.legend()
ax.grid(True)

```



```
[20]: import pandas as pd

df = pd.DataFrame({"N":NFEM,"error N":ErrorN},index=npoints)

df
```

```
[20]:
```

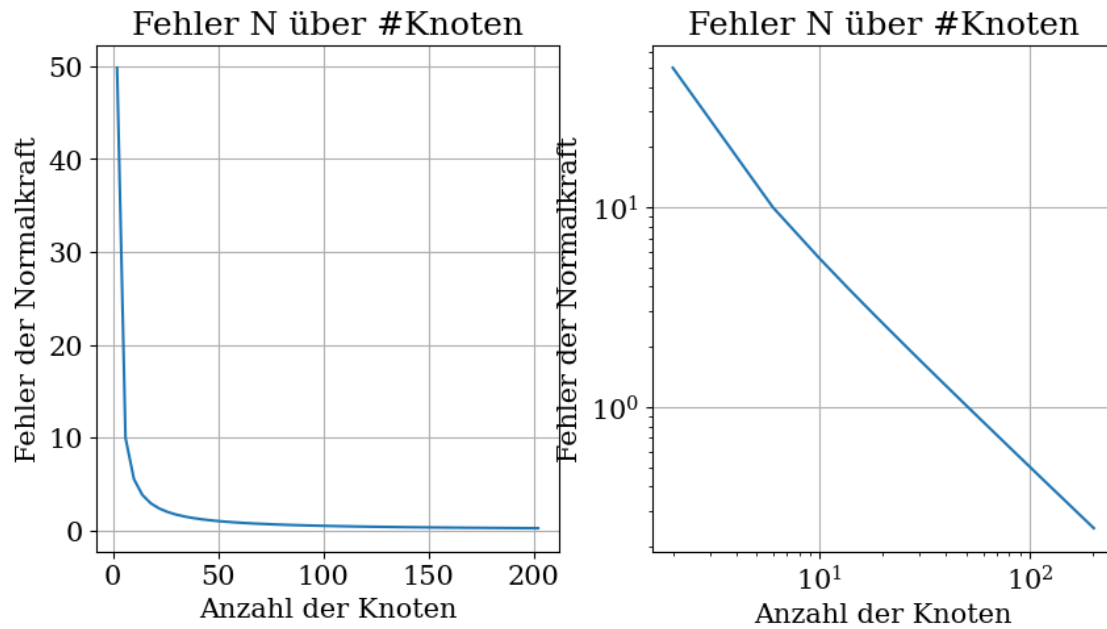
	N	error N
2.0	6.730000e+05	49.813572
6.0	1.207400e+06	9.962714
10.0	1.266778e+06	5.534841
14.0	1.289615e+06	3.831813
18.0	1.301706e+06	2.930210
22.0	1.309190e+06	2.372075
26.0	1.314280e+06	1.992543
30.0	1.317966e+06	1.717709
34.0	1.320758e+06	1.509502
38.0	1.322946e+06	1.346313
42.0	1.324707e+06	1.214965
46.0	1.326156e+06	1.106968

50.0	1.327367e+06	1.016604
54.0	1.328396e+06	0.939879
58.0	1.329281e+06	0.873922
62.0	1.330049e+06	0.816616
66.0	1.330723e+06	0.766363
70.0	1.331319e+06	0.721936
74.0	1.331849e+06	0.682378
78.0	1.332325e+06	0.646930
82.0	1.332753e+06	0.614982
86.0	1.333141e+06	0.586042
90.0	1.333494e+06	0.559703
94.0	1.333817e+06	0.535630
98.0	1.334113e+06	0.513542
102.0	1.334386e+06	0.493204
106.0	1.334638e+06	0.474415
110.0	1.334872e+06	0.457005
114.0	1.335088e+06	0.440828
118.0	1.335291e+06	0.425757
122.0	1.335479e+06	0.411682
126.0	1.335656e+06	0.398509
130.0	1.335822e+06	0.386152
134.0	1.335977e+06	0.374538
138.0	1.336124e+06	0.363603
142.0	1.336262e+06	0.353288
146.0	1.336393e+06	0.343542
150.0	1.336517e+06	0.334319
154.0	1.336634e+06	0.325579
158.0	1.336745e+06	0.317284
162.0	1.336851e+06	0.309401
166.0	1.336952e+06	0.301900
170.0	1.337047e+06	0.294755
174.0	1.337139e+06	0.287940
178.0	1.337226e+06	0.281433
182.0	1.337309e+06	0.275213
186.0	1.337389e+06	0.269263
190.0	1.337466e+06	0.263564
194.0	1.337539e+06	0.258101
198.0	1.337609e+06	0.252861
202.0	1.337677e+06	0.247829

```
[21]: fig, ax = plt.subplots(1,2,figsize=(10,5))
df["error N"].plot(ax=ax[0],title="Fehler N über #Knoten",xlabel="Anzahl der_
↪Knoten",ylabel="Fehler der Normalkraft")
ax[0].grid(True)

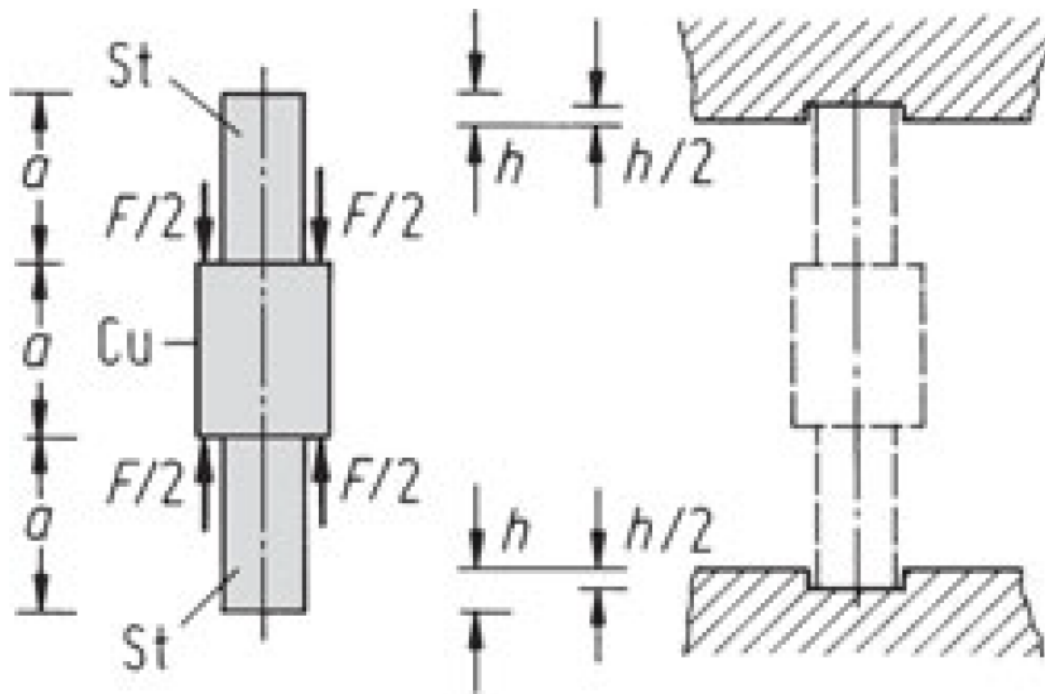
df["error N"].plot(ax=ax[1],logy=True,logx=True,title="Fehler N über_
↪#Knoten",xlabel="Anzahl der Knoten",ylabel="Fehler der Normalkraft")
```

```
ax[1].grid(True)
```



4.3 Beispiel 3 - Stabsystem

Der dargestellte Verbundstab soll zwischen zwei feste Wände geklemmt werden. Für den Einbau wird das mittlere Stabteil mit der Kraft F zusammengedrückt. Wie groß muss F mindestens sein, damit der Einbau gelingt? Wie groß sind die Spannungen im Stab nach dem Einbau? Um wieviel ist das Mittelstück nach dem Einbau kürzer als vor dem Einbau?



Gegebene Größen - $E\text{-Stahl} = 210 \text{ GPa}$ - $E\text{-Cu} = 105 \text{ GPa}$ - $A\text{-Stahl} = 30 \text{ mm}^2$ - $A\text{-Cu} = 60 \text{ mm}^2$ - $a = 150 \text{ mm}$ - $h = 3 \text{ mm}$

Gesucht - F um die Welle einzusetzen - Spannungen im Stab nach dem Einbau - Längenänderung des mittleren Wellenteils nach dem Einbau

4.3.1 Welche Kraft ist notwendig um den Stab einzubauen?

```
[22]: #####
# SetUp des Problems
#####

a = 150
Astahl = 30
Acu = 60
EStahl = 210*10**3
ECu = 105*10**3
h = 3

bsp3 = StabFEM(numnp=4,numel=3)
X = np.array([
    [0,0],
    [a,0.0],
    [2*a,0],
    [3*a,0],
])
```

```

elements = np.array([
    [0,1],
    [1,2],
    [2,3]
])
areas = [Astahl,Acu,Astahl]
youngsM = [ESTahl,ECu,ESTahl]
loads = [0,0,0]

# Setzen der Knotenkoordinaten, der Elementverbindungen und der Elementdaten
bsp3.setNodalCoordinates(X)
bsp3.setElementConnectivity(elements)
bsp3.setElementData(areas,youngsM,loads)

# Setzen der Randbedingungen
bsp3.setDirichletBoundaryCondition([0,1,1,2,2,3],[1,0,1,0,1,1],[0,h,0,-h,0,0])
    ↪ # Knoten, Richtung, Wert
# bsp3.setExternalForces([1],[0],[5000]) # Knoten, Richtung, Wert

# Kontrolle der Eingaben
print(f"Knotenkoordinaten:\n{bsp3.coords}")
print(f"Elementverbindungen:\n{bsp3.elements}")
print(f"Elementdaten:\n{bsp3.eData}")

```

Knotenkoordinaten:

```

[[ 0.  0.]
 [150. 0.]
 [300. 0.]
 [450. 0.]]

```

Elementverbindungen:

```

[[0 1]
 [1 2]
 [2 3]]

```

Elementdaten:

```

[{'area': 30, 'youngsmodulus': 210000, 'lineLoad': 0}, {'area': 60,
'youngsmodulus': 105000, 'lineLoad': 0}, {'area': 30, 'youngsmodulus': 210000,
'lineLoad': 0}]

```

[23]: bsp3.plotMesh()

[23]: (<Figure size 640x480 with 1 Axes>, <Axes: >)

```

/home/steffen/anaconda3/envs/rise/lib/python3.10/site-
packages/matplotlib/quiver.py:695: RuntimeWarning: divide by zero encountered in
scalar divide

```

```

    length = a * (widthu_per_lenu / (self.scale * self.width))

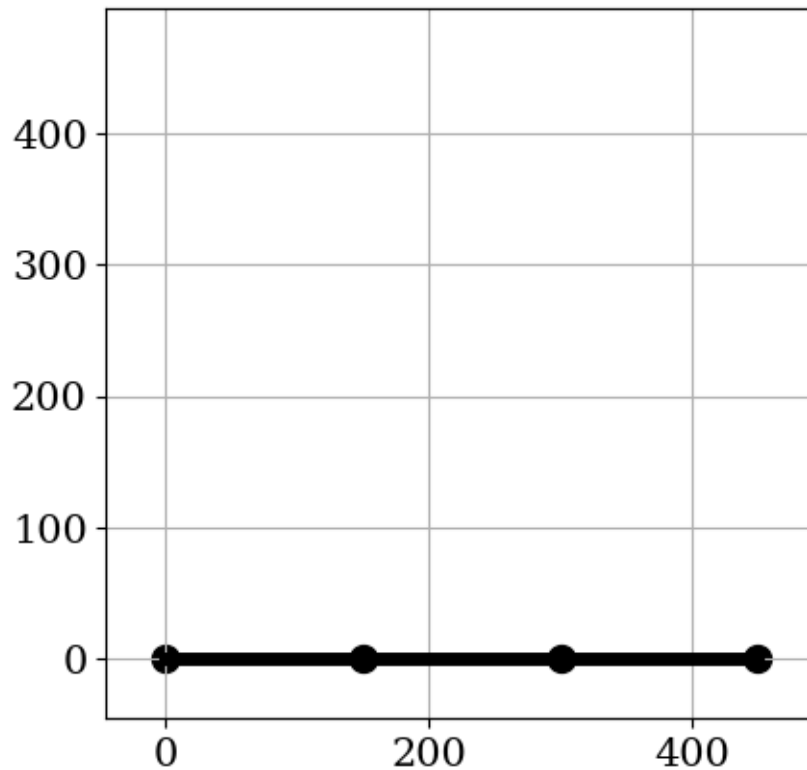
```

```

/home/steffen/anaconda3/envs/rise/lib/python3.10/site-
packages/matplotlib/quiver.py:695: RuntimeWarning: invalid value encountered in

```

```
multiply
length = a * (widthu_per_lenu / (self.scale * self.width))
```



```
[24]: #####
# Lösen des Problems
#####

# Schleife über alle Elemente, bilden der Elementmatrizen und -vektoren und
↳ assemblieren des globalen Systems

bsp3.assembleGlobalMatrix()
bsp3.assembleRightHandSide()
# Lösen des Gleichungssystems
bsp3.solveSystem()

display("Displacement:", bsp3.dof)
display("Force:", bsp3.Fges)
print(f'Kraft die notwendig ist, das mittlere Wellenstück zusammenzudrücken:↳
↳ {bsp3.Fges[1,0]}')
```

```
'Displacement:'
array([[ 3.,  0.]
```

```

[ 3.,  0.],
[-3.,  0.],
[-3.,  0.]]))

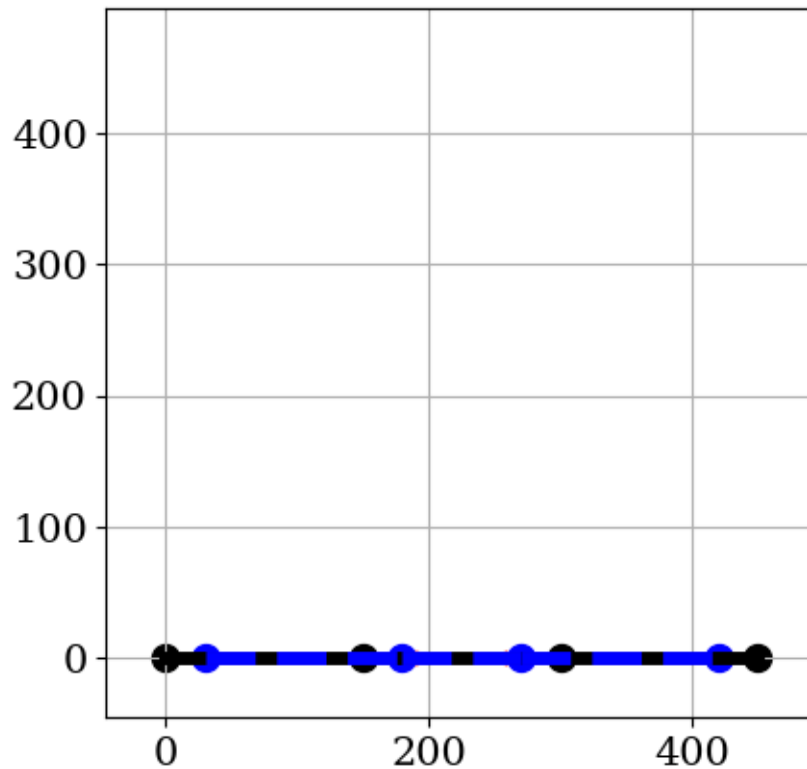
'Force:'
array([[ 0.,  0.],
       [252000.,  0.],
       [-252000.,  0.],
       [ 0.,  0.]])

Kraft die notwendig ist, das mittlere Wellenstück zusammenzudrücken: 252000.0

```

```
[25]: bsp3.plotMesh(deformed=True,scale=10)
```

```
[25]: (<Figure size 640x480 with 1 Axes>, <Axes: >)
```



4.3.2 Welche Spannung liegt nach dem Einbau vor

```

[26]: # Reset des Systems
      bsp3.resetFEM()

      # Setzen der Randbedingungen

```

```

bsp3.setDirichletBoundaryCondition(
    [0,0,1,2,3,3], # Knoten
    [0,1,1,1,0,1], # Richtung
    [h/2,0,0,0,-h/2,0]) # Wert

#####
# Lösen des Problems
#####

# Schleife über alle Elemente, bilden der Elementmatrizen und -vektoren und
↳ assemblieren des globalen Systems

bsp3.assembleGlobalMatrix()
bsp3.assembleRightHandSide()
# Lösen des Gleichungssystems
bsp3.solveSystem()

display("Displacement:", bsp3.dof)
display("Force:", bsp3.Fges)

```

'Displacement:'

```

array([[ 1.5,  0. ],
       [ 0.5,  0. ],
       [-0.5,  0. ],
       [-1.5,  0. ]])

```

'Force:'

```

array([[ 42000.,  0.],
       [  0.,  0.],
       [  0.,  0.],
       [-42000.,  0.]])

```

```

[27]: XN,N =bsp3.computeNormalkraft()
fig,ax = plt.subplots(1,1)

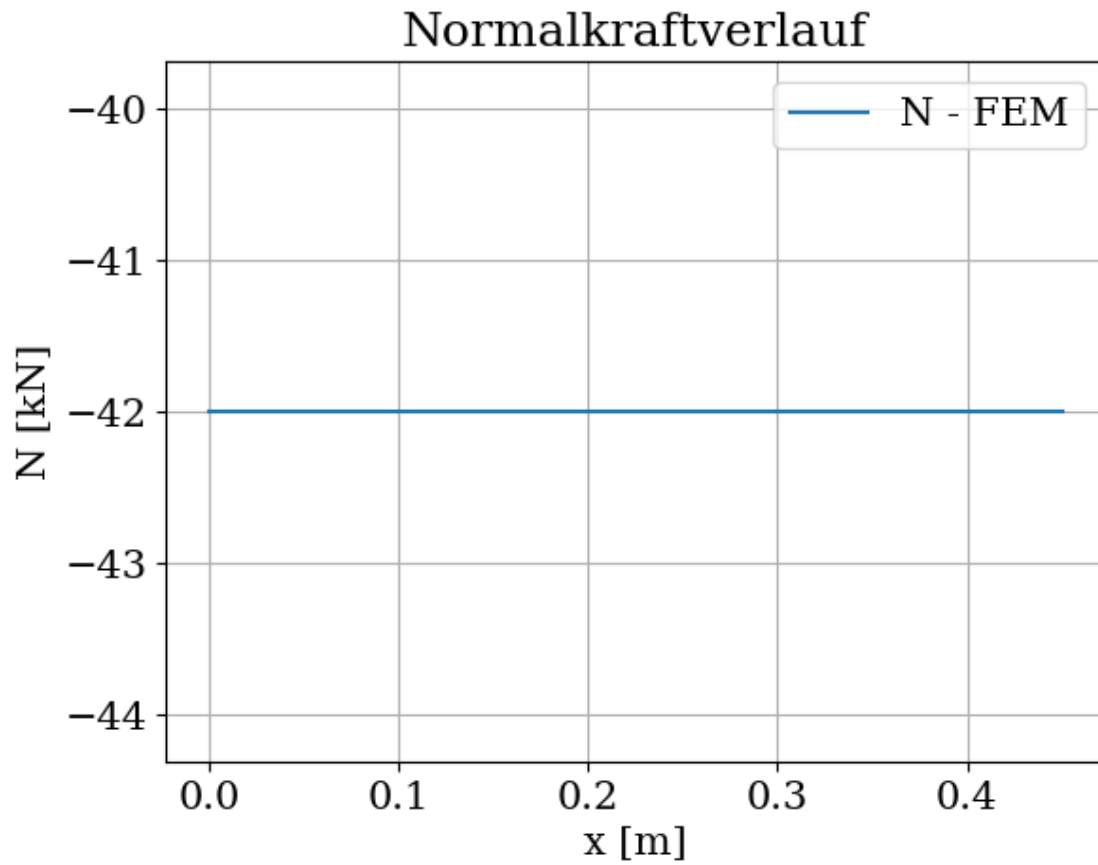
ax.plot(XN/1000,N/1000,label="N - FEM")
ax.set_title("Normalkraftverlauf")
ax.set_xlabel("x [m]")
ax.set_ylabel("N [kN]")
ax.grid(True)

ax.legend()
sigstahl = N[0]/(bsp3.eData[0]['area'])
sigcu    = N[0]/(bsp3.eData[1]['area'])
print(f'Spannungen im Stahlstab: {sigstahl}')
print(f'Spannungen im Kupferstab: {sigcu}')

```

Spannungen im Stahlstab: -1400.0

Spannungen im Kupferstab: -700.0

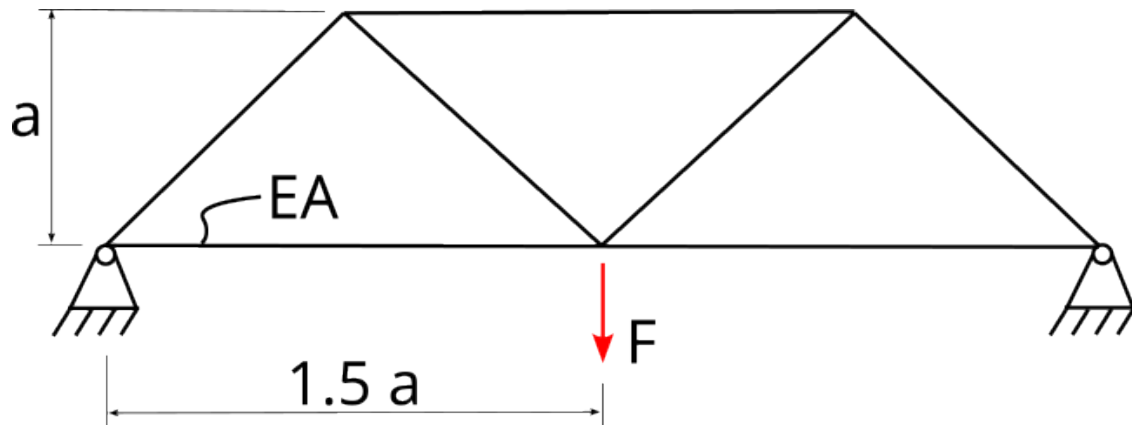


4.3.3 Dehnung im mittleren Stabteil

```
[28]: eps = (bsp3.dof[2,0]-bsp3.dof[1,0])/(a)
      print(f'Dehnung im mittleren Stab: {eps}')
```

Dehnung im mittleren Stab: -0.006666666666666667

4.4 Beispiel 4



Gegebene Größen

- E-Stahl = 210 GPa
- A = 33.4 cm² (I-Profil - 200)
- a = 5 m
- F = 10 kN

Gesucht

- Verschiebung im Kraftangriffspunkt

```
[29]: #####  
# SetUp des Problems  
#####  
  
a = 5*1000  
A = 33.4*10**2  
E = 210*10**3  
F = -1000*10**3  
  
X = np.array([  
    [0,0],  
    [1.5*a,0.0],  
    [2*1.5*a,0],  
    [0.5*1.5*a,a],  
    [0.5*1.5*a+1.5*a,a],  
])  
elements = np.array([  
    [0,1],  
    [1,2],  
    [0,3],  
    [3,1],  
    [1,4],  
    [4,2],
```

```

        [3,4]
    ])

    numnp = X.shape[0]
    numel = elements.shape[0]

    bsp4 = StabFEM(numnp,numel)

    areas = [A for i in range(0,numel)]
    youngsM = [E for i in range(0,numel)]
    loads = [0 for i in range(0,numel)]

    # Setzen der Knotenkoordinaten, der Elementverbindungen und der Elementdaten
    bsp4.setNodalCoordinates(X)
    bsp4.setElementConnectivity(elements)
    bsp4.setElementData(areas,youngsM,loads)

    # Setzen der Randbedingungen
    bsp4.setDirichletBoundaryCondition([0,0,2,2],[0,1,0,1],[0,0,0,0]) # Knoten, ⬇
    # ↪ Richtung, Wert
    bsp4.setExternalForces([1],[1],[F])

    # Kontrolle der Eingaben
    print(f"Knotenkoordinaten:\n{bsp4.coords}")
    print(f"Elementverbindungen:\n{bsp4.elements}")
    print(f"Elementdaten:\n{bsp4.eData}")

```

Knotenkoordinaten:

```

[[ 0.  0.]
 [7500. 0.]
 [15000. 0.]
 [ 3750. 5000.]
 [11250. 5000.]]

```

Elementverbindungen:

```

[[0 1]
 [1 2]
 [0 3]
 [3 1]
 [1 4]
 [4 2]
 [3 4]]

```

Elementdaten:

```

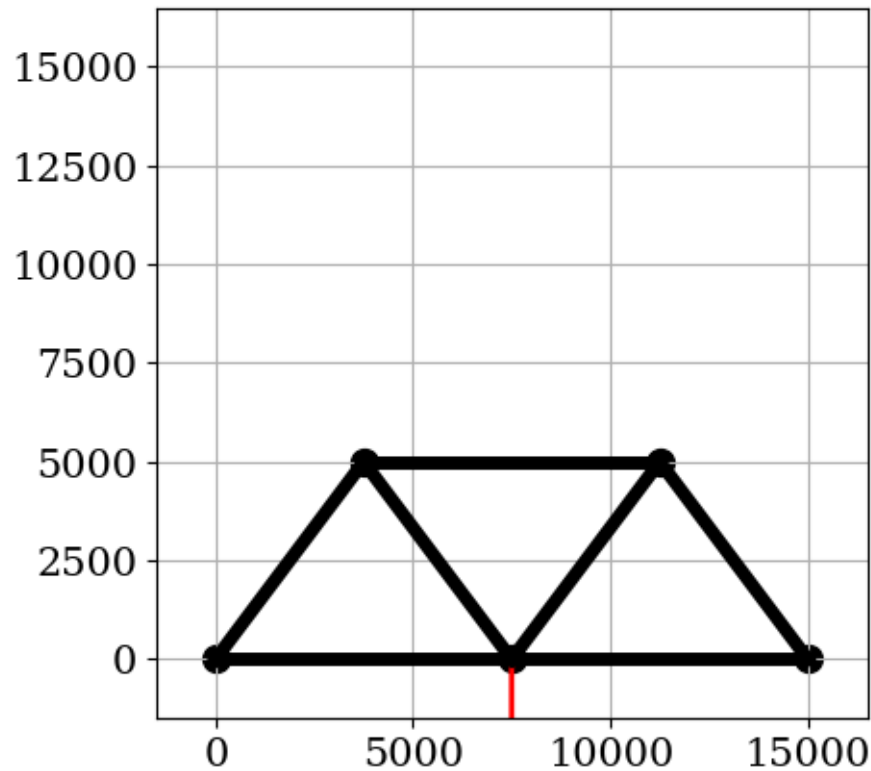
[{'area': 3340.0, 'youngsmodulus': 210000, 'lineLoad': 0}, {'area': 3340.0,
'youngsmodulus': 210000, 'lineLoad': 0}, {'area': 3340.0, 'youngsmodulus':
210000, 'lineLoad': 0}, {'area': 3340.0, 'youngsmodulus': 210000, 'lineLoad':
0}, {'area': 3340.0, 'youngsmodulus': 210000, 'lineLoad': 0}, {'area': 3340.0,
'youngsmodulus': 210000, 'lineLoad': 0}, {'area': 3340.0, 'youngsmodulus':

```

```
210000, 'lineLoad': 0}]
```

```
[30]: bsp4.plotMesh()
```

```
[30]: (<Figure size 640x480 with 1 Axes>, <Axes: >)
```



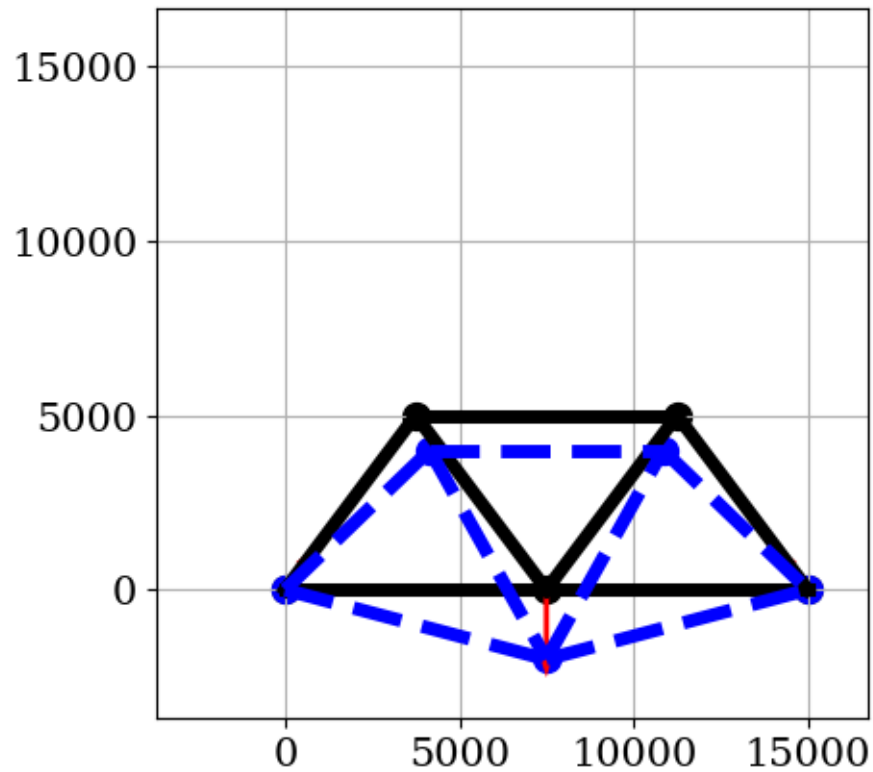
```
[31]: #####
# Lösen des Problems
#####

# Schleife über alle Elemente, bilden der Elementmatrizen und -vektoren und
↳ assemblieren des globalen Systems
bsp4.assembleGlobalMatrix()
bsp4.assembleRightHandSide()
# Lösen des Gleichungssystems
bsp4.solveSystem()
print(f'Verschiebung im Kraftangriffspunkt: {bsp4.dof[1,1]} mm')
```

Verschiebung im Kraftangriffspunkt: -19.93780296549757 mm

```
[32]: bsp4.plotMesh(deformed=True,scale=100)
```

```
[32]: (<Figure size 640x480 with 1 Axes>, <Axes: >)
```



Berechnungsablauf der Finiten Elemente Methode

1. Diskretisierung des Problems:
 - Knoten und Elemente definieren
2. Materialeigenschaften zuweisen
3. Randbedingungen aufbringen
4. Gleichungssystem assemblieren
5. Gleichungssystem lösen
6. Ergebnisse auswerten - Postprocessing